

Radiance Backend: Implementation Report

From a phone video to a simulated flight, and from a phrase to a goal

Suhan — Team Radiance, Department of Computer Science and Engineering,
University of Moratuwa

6 October 2026

Abstract

This report documents the backend of the Radiance project as implemented on the lab host *intellisense08* (RTX 2080, 8 GB): the resumable pipeline that turns a phone video into a 3D Gaussian Splat (3DGS) and flies an expert drone through it with Stanford MSL’s FiGS / SOUS-VIDE, the semantic layer that attaches CLIP and DINOv2 features to the frozen splat so that a phrase resolves to a 3D goal, and the Galley web console that queues and inspects all of it. For every pipeline step we state exactly which FiGS, nerfstudio or third-party call does the work. We then report the Phase 5 comparison of four semantic variants on two scenes — the training-free lift localises best (0.62 and 0.63 top-1, ≈ 0.2 m median error) and the FMGS feature field with its paper weights the worst — explain what the numbers mean, list concrete improvements, and assess whether embedding semantics *while training* the splat would be better than embedding them into a frozen one. Our conclusion is that, for a simulator whose policies and courses are tied to one fixed splat, the frozen route is the right default, and joint training is worth testing only as a separate experiment with compressed features.

Contents

1	System overview	2
2	Capture to flight: <code>figs_pipeline.py</code>	2
2.1	Splat training	4
2.2	Courses, preview and timing	4
2.3	SV-Net student: <code>svnet_pipeline.py</code>	4
3	Semantics on the frozen splat: <code>semantic_pipeline.py</code>	5
3.1	Teachers	5
3.2	Backend 1: the lift (no training)	5
3.3	Backend 2: FMGS on the frozen splat	5
3.4	The table and the query	6
4	The Galley backend	6
5	Results: four semantic variants on two scenes	6
6	What can be improved	7
7	Assessment: semantics during splat training, or on a frozen splat?	8

1 System overview

The backend has three layers (Figure 1):

1. **Pipelines** (Python scripts in `figs/`), each a sequence of resumable steps: `figs_pipeline.py` (capture → splat → course → expert flight), `svnet_pipeline.py` (expert rollouts → SV-Net student), and `semantic_pipeline.py` (teachers → lift or FMGS → feature table). Each step writes a completion marker with a fingerprint of the parameters it used, so an interrupted run restarts at the failed step and a changed parameter invalidates exactly the steps downstream of it.
2. **Libraries**: FiGS / SOUS-VIDE (upstream, unmodified except documented submodule patches), nerfstudio 1.1.4 (`splatfacto`), gsplat 1.0.0, hloc + COLMAP, acados, and our own `radiance_semantics` package.
3. **Galley** (`ui/backend/galley`, FastAPI + SQLite; React frontend): a one-job-at-a-time GPU queue that runs the pipelines, plus fast CPU tools (course preview, splat export) and a long-lived CPU worker that answers semantic queries.

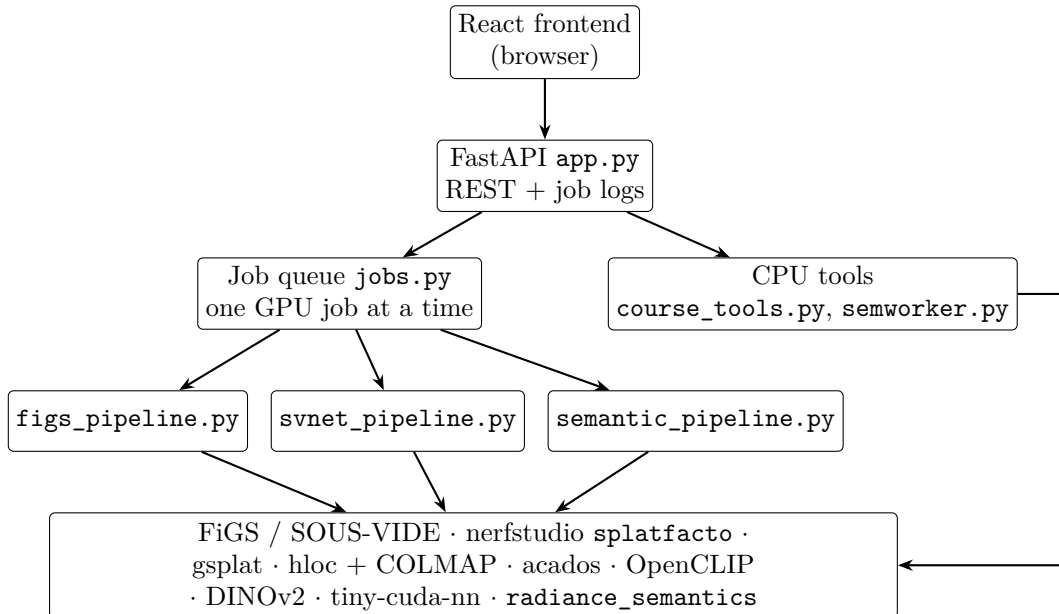


Figure 1: Backend layers. Every GPU-heavy action is a job in the queue; previews and semantic queries run on the CPU beside it.

Environment. Every job runs as `bash -c 'source figs_env.sh && exec ...'` in a fresh shell, because the pipelines need the kitchen conda environment plus `ACADOS_SOURCE_DIR`, `LD_LIBRARY_PATH` and `PYTHONNOUSERSITE=1`, which cannot be fixed inside a running process. The stack is pinned (torch 2.1.2/CUDA 11.8, nerfstudio 1.1.4, gsplat 1.0.0, tiny-cuda-nn 2.0 built for sm_75); additions go in with `--no-deps` against a frozen constraints file. Each job gets its own process group, so cancelling reaches `ns-train`, COLMAP and ffmpeg children too.

2 Capture to flight: `figs_pipeline.py`

Table 1 lists the steps and, for each, what actually does the work. Steps marked *FiGS* call upstream code unchanged; our code around them adds staging, checks, logging, VRAM monitoring and diagnostics for known failure messages.

Table 1: Steps of `figs_pipeline.py` and the calls behind them.

Step	What it does	What it calls
<code>preflight</code>	Environment, imports, <code>acados</code> libraries, GPU, free disk.	Imports <code>torch</code> , <code>nerfstudio</code> , <code>gsplat</code> , <code>hloc</code> , <code>figs</code> , <code>acados_template</code> , <code>pycolmap</code> , <code>cv2</code> , <code>open3d</code> ; <code>ctypes</code> load of <code>libacados.so</code> ; <code>nvidia-smi</code> . <code>ffprobe</code> .
<code>probe</code>	Inspects the source video (codec, size, HDR, VFR, frame count).	
<code>transcode</code>	Stages the video into <code>gsplats/capture/<scene>.mp4</code> at 1920×1080 , fixed fps.	<code>ffmpeg -vf scale=W:H,format=yuv420p -r fps -c:v libx264 -crf 18</code> . (No HDR tone mapping: HLG footage stays washed out.)
<code>aruco</code>	Confirms the printed marker is visible and when; 10s histogram; hard-fails if the ID never appears.	OpenCV <code>cv2.aruco.ArucoDetector</code> with <code>DICT_4X4_50</code> , every 2nd frame.
<code>config</code>	Writes the FiGS capture config.	<code>configs/captures/<scene>.json</code> : <code>camera</code> : <code>null</code> (use SfM intrinsics), <code>extractor</code> : <code>num_images</code> , <code>num_marked</code> , <code>marker_length</code> , <code>marker_id</code> .
<code>patch</code>	Re-applies small fixes to <code>hloc</code> submodule files for the installed <code>pycolmap</code> API.	Edits <code>Hierarchical-Localization/hloc/{reconstruction,extract_features}.py</code> only when needed.
<code>sfm</code>	Frames $\rightarrow$ structure from motion $\rightarrow$ metric alignment.	FiGS <code>figs.render.capture_generation.generate_gspat(scene)</code> , run with a <code>subprocess</code> shim that defers its final <code>ns-train</code> call to the next step. Inside: <code>extract_frames</code> (sharp frames), <code>nerfstudio ImagesToNerfstudioDataset</code> (<code>hloc SuperPoint</code> + <code>SuperGlue</code> , exhaustive pairs, <code>COLMAP</code>), <code>extract_positions</code> (<code>ArUco solvePnP</code> with <code>SOLVEPNP_IPPE_SQUARE</code> on frames where the marker is the only one), <code>compute_ransac_transform</code> (similarity: scale, rotation, translation), then <code>transforms.json</code> and <code>sparse_pc.ply</code> rewritten in metres.
<code>train</code>	Trains the splat; refuses a second model per scene unless told to archive.	<code>ns-train splatfacto --pipeline.model.camera-optimizer.mode S03xR3 nerfstudio-data --orientation-method none --center-method none --auto-scale-poses False</code> (+ iterations, downscale, <code>cache-images</code>).
<code>verify</code>	Registration rate, point cloud, exactly one model.	<code>transforms.json</code> , <code>open3d</code> on <code>sparse_pc.ply</code> .
<code>bounds</code>	Camera box and recommended waypoint box; course frame.	NumPy over camera positions; course frame = $(x, -y, -z)$ of the splat frame (z down).
<code>course</code>	Validates the course file: integer cells, keyframe timing, keyframes inside the capture.	Our checks (<code>course_int_cells</code> , <code>course_fast_segments</code> , <code>course_inside</code>). FiGS's <code>KF_to_TpF0</code> reads an integer cell as the previous cell's value, so integers are refused.

Step	What it does	What it calls
simulate	Flies the expert and renders the onboard camera.	FiGS Simulator (scene, method, frame), VehicleRateMPC (pilot, course, frame) (its constructor runs MinTimeSnap and TsFO_to_tXU , then builds the acados MPC), sim.simulate (ctl, tA, tB, x0), generate_videos.images_to_mp4 . Tracking error and pixel statistics are computed here.
validate	Rejects a garbage MP4 (wrong size, black frames, too short).	ffprobe , frame statistics.
record	Writes the run record.	SousVide/runs/<scene>_<ts>.json (all step results).

2.1 Splat training

splatfacto (nerfstudio’s 3DGS) trains for 30,000 iterations from the SfM points. Three choices matter downstream:

- **Frames.** Orientation and centring are **none** and auto-scaling is off, so the splat lives in exactly the metric frame of **transforms.json**. Courses, the simulator and the semantic table all use that frame (the course frame flips y and z).
- **Camera optimiser S03xR3:** per-camera pose corrections learnt during training. Splatfacto applies them only in training mode; we apply them explicitly when we need the training views (Section 3). On backroom they are worth +4.22 dB PSNR (31.68 vs 27.46 dB).
- **One model per scene.** FiGS loads the single model under **outputs/<scene>/**; retraining archives the old one, which also changes the key that semantic tables and Galley’s **.splat** cache are tied to.

Reference cost on the RTX 2080: 30k iterations in ≈ 34 min at 960×540 with a peak of 5.2 GB (2.4 GB with **--cache-images cpu**). Backroom has 532,361 Gaussians; flightroom 379,684.

2.2 Courses, preview and timing

A course is a list of keyframes, each with a time t and a 4×5 matrix of flat outputs (x, y, z , yaw and their derivatives; empty cells are free). Galley’s course editor previews it on the CPU with **course_tools.py preview**, which runs the same two FiGS calls the MPC constructor makes (**MinTimeSnap**, **TsFO_to_tXU**) with the expert’s **kT**, **hz**, bounds and the frame’s mass and thrust coefficient, then checks clearance against the SfM points (gap = distance to the 5th-nearest point minus the 0.19 m body radius) and the capture volume.

With the expert Viper (**kT** = 10) the file’s times are only a starting guess: SLSQP re-optimises the segment durations. We found (6 Oct) that this optimisation, run on raw durations, cannot recover from a badly squeezed guess — snap cost grows like $1/\Delta t^7$ — and stops at the guess while reporting success, so such a course flies as previewed (one test: 33 m/s, 528 % of the thrust limit). The editor’s *Add* now inserts time in proportion to distance, and the editor, the lint endpoint and the **course** step flag legs faster than 5 m/s; a re-time that returns the file’s times while breaking limits is reported as stalled.

2.3 SV-Net student: **svnet_pipeline.py**

Steps **rollout** (expert flights with SOUS-VIDE’s data generation over the cohort’s courses), **observe** (per-pilot observations), **train_hist** and **train_comm** (the history and command networks), and **deploy** (closed-loop flight of the student in the splat). The student learns from renders of one specific splat, which is why the splat must stay frozen once a cohort exists.

3 Semantics on the frozen splat: `semantic_pipeline.py`

Table 2: Steps of `semantic_pipeline.py` (backends: `lift`, `fmgs`, `fmgs_c`).

Step	What it does	What it calls
<code>preflight</code>	Imports, GPU, active run, time and disk estimate.	<code>radiance_semantics.env_check</code> , <code>paths.find_scene_run</code> .
<code>cameras</code>	Refined vs raw pose render check.	<code>nerfstudio eval_setup</code> (<code>cameras.load_pipeline</code> , run from <code>gsplats/workspace</code>), <code>CameraOptimizer.apply_to_camera</code> via <code>cameras.optimized_c2w</code> , <code>splatfacto</code> rendering; PSNR.
<code>teachers</code>	2D features per photo, cached as fp16.	OpenCLIP ViT-B/16 (LAION-2B) on a 7-scale crop pyramid (LERF), averaged onto one grid ($\approx 71 \times 40$); DINOv2 ViT-S/14 via <code>torch.hub</code> (patch tokens, $\approx 64 \times 36$).
<code>lift</code>	Blend-weighted average of teacher features onto each Gaussian.	<code>gsplat 1.0.0 rasterization</code> in 32-channel chunks (its backward limit); two backward passes per view give numerator and weights.
<code>export</code>	Table in Galley’s <code>.splat</code> order.	<code>course_tools.splat_order</code> , <code>store.write_table</code> .
<code>fmgs / fmgs_c</code>	Train the FMGS feature field against the frozen Gaussians (<code>fmgs_c</code> : CLIP only).	<code>radiance_semantics.fmgs.train</code> : <code>tiny-cuda-nn</code> hash grid (split into two 12-level grids on <code>sm_75</code>) + CutlassMLP heads, <code>gsplat</code> feature rendering, PyTorch Adam with dynamic loss scaling.
<code>bake / bake_c</code>	Field evaluated at every Gaussian $\rightarrow$ table.	<code>fmgs.bake</code> , unseen rows from the <code>lift</code> ’s weights.

3.1 Teachers

For each training photo, CLIP is applied to square crops at 7 scales (5–50 % of the short side), each scale’s grid of unit embeddings is resampled onto a common grid and the scales are averaged (LERF/FMGS). DINOv2 supplies dense patch features that delineate objects but carry no language. Backroom: 300 frames, 3,529 crops per frame, 31.5 min, 1.3 GB.

3.2 Backend 1: the lift (no training)

For Gaussian i with blending weight $w_i(v, p)$ at pixel p of view v and teacher map F_v :

$$\mathbf{f}_i = \frac{\sum_{v,p} w_i(v, p) F_v(p)}{\sum_{v,p} w_i(v, p)}.$$

Rendering is linear in the features, so the numerator is the gradient of $\sum_p \langle \text{render}(\mathbf{f}), F_v \rangle$ at $\mathbf{f} = 0$ and the denominator the gradient of $\sum_p \text{render}(\mathbf{1})$: no optimisation, 3–4 min per scene.

3.3 Backend 2: FMGS on the frozen splat

A multi-resolution hash grid (24 levels, $16 \rightarrow 512$, 2^{20} entries, 8 features; built as two 12-level `tiny-cuda-nn` grids because a 192-d grid does not launch on `sm_75`) feeds two MLP heads (CLIP 512, DINO 384). Each of 4,200 steps renders the field’s outputs for the trainable Gaussians (the

40 % most opaque, in frustum) from one training view at 480×270 and minimises $0.2 \text{Huber}(\text{CLIP}) + 0.8 \|\text{DINO}\|^2 + 0.01 \mathcal{L}_{\text{PA}}$. The Gaussians never enter the optimiser and their checksum is verified. Two engineering fixes made it fit and train on 8 GB: PyTorch’s allocator with expandable segments plus an out-of-memory retry, and dynamic loss scaling with a norm floor in the pixel-alignment term to stop fp16 underflow/overflow inside tiny-cuda-nn. Cost: 31 min (F-CD) and 14 min (F-C) on backroom.

3.4 The table and the query

Both backends write the same per-Gaussian table (unit CLIP and DINO rows, blend weight, geometry, PCA colours), row i = record i of Galley’s `.splat`, keyed to the checkpoint. A long-lived CPU worker (`semworker.py`, JSON lines over stdin/stdout, `CUDA_VISIBLE_DEVICES=""`) keeps the CLIP text tower and memory-mapped tables loaded and answers a query in ≈ 0.5 – 0.9 s: LERF relevancy against canonical negatives, a threshold relative to the query’s own peak ($\tau = 0.55 + 0.5(\text{peak} - 0.55)$), 0.1 m voxel connected components, ranking, and an approach point 1 m from the object towards where the cameras looked from, clipped to the flyable box and checked for clearance. The splat editor sends the result to a course as its final keyframe.

4 The Galley backend

Module	Role
<code>app.py</code>	FastAPI routes: scenes, models (archive/promote), jobs and logs, courses (save, lint, preview, geometry, <code>.splat</code>), semantics (status, query, relevancy bytes, PCA, annotations, evaluation), videos/uploads, Drive import. Token-protected, loopback only.
<code>jobs.py</code>	The queue: strictly one job at a time in submission order, fresh shell with <code>figs_env.sh</code> , own process group, live log stored in SQLite, interrupted jobs marked on restart.
<code>pipeline.py</code> , <code>svnet.py</code> , <code>semantics.py</code>	Pydantic models that validate a run and build the exact command line (<code>build_argv</code>); read step markers, results and run records; status per scene (e.g. semantic tables stale when the checkpoint key changes).
<code>course.py</code>	CourseTools : runs <code>course_tools.py</code> (geometry, preview, splat export) in the kitchen environment on the CPU; integer-cell and timing lint.
<code>semworker.py</code> <code>configs.py</code>	Starts, restarts and talks to the semantic query worker. Validated read/write of FiGS config files (courses as floats in the upstream layout, mirrored to the repo overlay).
<code>videos.py</code> , <code>drive.py</code>	Resumable uploads and Google Drive import of capture videos.
<code>tfevents.py</code> , <code>db.py</code> , <code>settings.py</code>	TensorBoard curves, SQLite store, machine profiles (<code>ui/machines/<host>.toml</code>).

5 Results: four semantic variants on two scenes

Query sets were annotated from the training photos and frozen in git before any query ran: backroom 13 objects (incl. synonyms and a 3-instance class) + 3 absent objects; flightroom 16 + 3.

A query is a *hit* when its top candidate’s box (+0.3 m) holds an annotated instance or its centroid is within 0.75 m. The query settings were fixed on a separate development set.

Scene	Variant	Top-1 hit	Err. med. (m)	Neg. FP	Neg. AUROC	Build (min)	VRAM (GB)
backroom	L-C	8/13	0.19	1/3	0.85	3.0	7.4
	L-CD	8/13	0.19	1/3	0.85	3.0	7.4
	F-C	6/13	0.32	1/3	0.80	13.9	5.8
	F-CD	7/13	0.20	1/3	0.80	31.4	7.8
flightroom	L-C	10/16	0.22	1/3	0.90	4.4	2.4
	L-CD	10/16	0.21	0/3	0.90	4.4	2.4
	F-C	8/16	0.11	0/3	0.81	11.5	5.1
	F-CD	4/16	0.18	0/3	0.96	25.2	7.5

Table 3: L = lift, F = FMGS; C = CLIP only, CD = CLIP + DINO (query-time DINO for L-CD, DINO loss for F-CD). The shared teacher extraction (31–50 min) is not included in Build. Queries take 0.54–0.9 s on the CPU for every variant.

What the numbers mean.

- **The training-free lift is the best localiser here.** It finds 62–63 % of objects with ≈ 0.2 m median error, in 3–4 min. Averaging measured features over every view that saw a Gaussian is hard to beat when the geometry is already good.
- **FMGS with CLIP alone (F-C) localises tightly but finds fewer objects:** its smooth field sharpens what it does find (0.11 m on flightroom) but blurs weak, small or thin objects below the threshold.
- **FMGS with its paper weights (F-CD) fails on flightroom (4/16)** because its CLIP channel is spatially fragmented: for “red cup” the Gaussians above threshold form 61 clusters of ≤ 3 , none large enough to count. With unit-norm 512-d CLIP targets, the 0.2-weighted CLIP term is about four orders of magnitude below the 0.8-weighted DINO term, so DINO shapes the shared hash grid and the CLIP head is noisy at Gaussian scale.
- **DINO at query time (L-CD) changes no hit;** it removes one false positive on an absent object at +50–70 ms per query. Lifted DINO rows are smooth (neighbour similarity 0.945), so there is little contrast to diffuse or split on.
- **The failures are mostly recognition, not geometry.** In every variant, “no candidate” (peak relevancy below the 0.55 floor) dominates: armchair, keyboard, floor lamp, tripod — thin, low-texture or generic objects that CLIP does not single out. Wrong-object errors are colour + material phrases with a similar object nearby (“purple foam mat”).
- **Sensitivity.** Only the relevancy floor moves hits (0.50: +1–2 hits per scene but twice the false positives); `rel_alpha`, `cluster_voxel` and the lift’s render width (480/720/960 px) change nothing material — the teacher grids are only ≈ 71 cells wide, so 480 px already oversamples them $\approx 7\times$.

Caveats. 29 positive and 6 negative queries over two scenes, one training seed per FMGS variant, ground truth placed from photos (validated at 0.09–0.42 m against hand placements). A third scene (GTN lab) was captured but its SfM folded two similar walls into one, so it was excluded.

6 What can be improved

1. **Better teachers before better fusion.** Most misses are CLIP recognition failures. A larger CLIP (ViT-L/14) or a region-level model (e.g. SAM masks pooled into CLIP, as in LangSplat)

would give sharper, object-shaped teacher maps; the lift would carry that improvement straight into 3D.

2. **Rebalance FMGS’s losses** (normalise each term by its magnitude, or raise the CLIP weight) and re-evaluate on a new development set; the paper weights do not transfer to unit-norm targets.
3. **Per-query absence decision.** A single global floor cannot separate absent objects (AUROC 0.80–0.96 says the peak relevancy carries the signal); a calibrated per-scene threshold or a “best match vs. background” ratio test would.
4. **Instance-aware clustering.** Voxel connected components merge touching objects and split thin ones; segment-level grouping (SAM in 2D, lifted) would fix both.
5. **Capture quality.** Tone-map HDR in `transcode`, keep the marker visible throughout, avoid ultra-wide lenses, and detect SfM folds (camera yaw histogram, depth consistency) automatically.
6. **Robust time optimisation.** Optimise log-durations instead of raw ones (converged in our test where FiGS’s SLSQP stalled) — upstream code, so proposed as a patch, not applied.
7. **More evaluation.** More scenes, multiple seeds, the Compare view for qualitative checks, and the Phase 6 test of whether language-generated courses fly as well as hand-made ones.

7 Assessment: semantics during splat training, or on a frozen splat?

The two routes. *Joint* methods learn a feature per Gaussian (or a feature field) in the same optimisation that fits colour and geometry, so densification and pruning also respond to the feature loss: Feature 3DGS [2] distils 2D foundation features this way (with a low-dimensional feature and a decoder to stay tractable), LEGaussians [3] likewise for language. *Staged* methods fit RGB first and then fit features with geometry fixed: LangSplat [4] trains compressed language features on a pre-trained scene, and FMGS [1] trains its hash-grid feature field after the Gaussians — the route we implemented. *Training-free* methods lift 2D features onto a fixed splat by its blending weights [5, 6]; LUDVIG and Occam’s LGS report results comparable to trained features at a fraction of the cost, which our Phase 5 lift results agree with.

What joint training could gain.

- Geometry that follows semantic boundaries: densification could add Gaussians where a small object’s features differ from its surroundings, which might help thin objects.
- A single training run instead of two, when a new scene is built anyway.
- Feature 3DGS reports that feature distillation can act as a mild regulariser for novel-view quality; the effect depends on the scene and the feature weight.

What it would cost in this project.

- **It breaks the frozen-splat contract.** Courses, clearance checks, the `.splat` cache and above all the SV-Net students are tied to one exact set of Gaussians. Any change to the semantics model would retrain the splat and invalidate cohorts; with a frozen splat, semantics can be rebuilt, compared and swapped at no cost to the simulator.
- **Risk to the photometric quality the policy trains on.** The feature loss competes with the colour loss for the same Gaussians and densification budget; any loss of RGB fidelity is a loss for SV-Net, whose input is the render.

- **Memory on 8 GB.** Splat training already peaks at 5.2 GB. Full 512+384-d features on 500k Gaussians are ≈ 1.8 GB of parameters, ≈ 7 GB with Adam state — impossible without compressing to a few dimensions (LangSplat’s 3-d autoencoder, Feature 3DGS’s decoder), which loses information our query uses.
- **Iteration speed and comparability.** The lift builds in 3–4 min and FMGS in 12–31 min; a joint run is a full 30k-iteration retrain per variant, and each variant would have different geometry, so a fair comparison of semantic backends would no longer be possible.
- **It does not address our main failure mode.** Phase 5’s misses are mostly CLIP not recognising the object at all (peak below threshold); better geometry does not create a teacher signal that is not there.

Verdict. For this system — a simulator whose value is a fixed, photometrically faithful splat that policies and courses are built on — **embedding semantics into the frozen splat is the better default**: it is cheaper, keeps every downstream artefact valid, allows backends to be compared on identical geometry, and our results show the simplest frozen method (the lift) already outperforms a trained field. Joint training is a reasonable *experiment*, not a replacement: on the planned nerfstudio branch, train a splatfacto variant with a low-dimensional semantic feature and decoder, and compare it on the same frozen query sets *and* on RGB PSNR, so that any gain in localisation is weighed against what it does to the images the drone learns from. Improving the teachers (Section 6, item 1) is likely to buy more than either.

References

- [1] X. Zuo, P. Samangouei, Y. Zhou, Y. Di, M. Li. *FMGS: Foundation Model Embedded 3D Gaussian Splatting for Holistic 3D Scene Understanding*. IJCV 2024.
- [2] S. Zhou et al. *Feature 3DGS: Supercharging 3D Gaussian Splatting to Enable Distilled Feature Fields*. CVPR 2024.
- [3] J.-C. Shi, M. Wang, H.-B. Duan, S.-H. Guan. *Language Embedded 3D Gaussians for Open-Vocabulary Scene Understanding*. CVPR 2024.
- [4] M. Qin, W. Li, J. Zhou, H. Wang, H. Pfister. *LangSplat: 3D Language Gaussian Splatting*. CVPR 2024.
- [5] J. Marrie et al. *LUDVIG: Learning-free Uplifting of 2D Visual Features to Gaussian Splatting Scenes*. 2024.
- [6] J. Cheng et al. *Occam’s LGS: An Efficient Approach for Language Gaussian Splatting*. 2024.
- [7] J. Kerr et al. *LERF: Language Embedded Radiance Fields*. ICCV 2023.